

TECH

# DEBUGGING

DEBUGGING

# DEBUGGING

20 ЗАДАЧ    7 ДНЕЙ    КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК    общая практика индустрии

# 20 ВОПРОСОВ      7 ДНЕЙ

|                   |                                                            |                    |                                                 |
|-------------------|------------------------------------------------------------|--------------------|-------------------------------------------------|
| ДЕНЬ 1<br>СРЕДНИЙ | ИНФИНИТНЫЙ RENDER<br>ЛАГ INPUT<br>Q1 · Q2 · Q3             | INFINITE EFFECT    | Три самых частых React-бага.                    |
| ДЕНЬ 2<br>ТЯЖЁЛЫЙ | WS FREEZE<br>RCH<br>Q4 · Q5 · Q6                           | STALE QUOTES       | STALE SEA Real-time проблемы и race conditions. |
| ДЕНЬ 3<br>ТЯЖЁЛЫЙ | MEMORY LEAK<br>OWN A11Y<br>Q7 · Q8 · Q9                    | MODAL KEYBOARD     | DROPD Утечки и accessibility-баги.              |
| ДЕНЬ 4<br>СРЕДНИЙ | TABLE FOCUS<br>-RENDER<br>Q10 · Q11 · Q12                  | REACT KEY          | CONTEXT RE Фокус, ключи, контекст-перформанс.   |
| ДЕНЬ 5<br>СРЕДНИЙ | useMemo НЕ ПОМОГАЕТ<br>PLAYWRIGHT FLAKY<br>Q13 · Q14 · Q15 | BUNDLE +500KB      | Performance и стабильность тестов.              |
| ДЕНЬ 6<br>ТЯЖЁЛЫЙ | HYDRATION MISMATCH<br>CORS<br>Q16 · Q17 · Q18              | PROD-ONLY ERROR    | SSR / build-only / network проблемы.            |
| ДЕНЬ 7<br>ТЯЖЁЛЫЙ | XSS<br>Q19 · Q20                                           | TOKEN REFRESH RACE | Безопасность и race в auth.                     |

РИТМ    30-60 МИН ТЕОРИЯ    15-30 МИН КОД В КОНСОЛИ    15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ    ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

**ИНФИНИТНЫЙ RENDER INFINITE EFFECT ЛАГ INPUT****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Три самых частых React-бага.

**ВОПРОСЫ ДНЯ**

- Q01 Компонент бесконечно re-render'ится – найди причину.
- Q02 useEffect вызывается бесконечно – исправь.
- Q03 Input лагает при фильтрации большого списка – исправь.

## Компонент бесконечно re-render'ится – найди причину.

### ОПИСАНИЕ

Бесконечный ре-рендер обычно вызывает `setState` внутри `render` или в `useEffect` без правильных `deps`. Также – нестабильная ссылка на объект/функцию в `deps` `useMemo` / `useCallback`. Дебаг через `React Profiler` + `console.log` в `render` с маркером.

### КАК РАБОТАЕТ

- 01 Проверить: `setState` в `render`? – bug.
- 02 `useEffect` без `deps` читает `state` и `setState` – loop.
- 03 Inline-объект как `deps` `useMemo` – каждый раз новая ссылка.
- 04 `Profiler` покажет частоту рендеров.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Симптом: вкладка зависает, `fps` падает.
- 02 `DevTools` → `Components` / `Profiler`.
- 03 `console.log('Comp render')` – счётчик растёт.

### ПОДВОДНЫЕ КАМНИ

- 01 Inline `{x}` в `deps` – новая ссылка.
- 02 `setState` без условия → бесконечно.
- 03 `useEffect` ↔ `useState` каскад без `break`-условия.

### МОГУТ ДОСПРОСИТЬ

- 01 Как сломать каскад?
- 02 Чем функциональный `setState` помогает?

### ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useState#avoiding-infinite-update-loop>

## ВОПРОС Q02

**useEffect вызывается бесконечно – исправь.**

## ОПИСАНИЕ

Чаще всего: deps содержат объект/функцию, которая пересоздается на каждый рендер. Решение – мемоизировать зависимость через `useMemo` / `useCallback`, или брать нужные значения примитивами, или использовать `ref` для стабильной ссылки.

## КАК РАБОТАЕТ

- 01 Inline-объект в deps → каждый рендер новый.
- 02 `useMemo` / `useCallback` стабилизируют.
- 03 Брать примитивы (`id`, `str`), не объект.
- 04 `ref` для редко меняющихся значений.

## ГДЕ ВСТРЕЧАЕТСЯ

- 01 `fetch` в `useEffect` → `setState` → перезапуск `useEffect`.
- 02 Подписка на `observable`, чья `value` – новый объект.

## ПОДВОДНЫЕ КАМНИ

- 01 Удалить deps → `eslint warning`, но `stale closure`.
- 02 `useCallback` без правильных deps – та же проблема.

## КОД

// Стабилизация

```
// плохо
useEffect(() => fetch(opts), [{ id }]);
// хорошо
useEffect(() => fetch({ id }), [id]);
```

## МОГУТ ДОСПРОСИТЬ

- 01 Что показывает `exhaustive-deps`?
- 02 Когда брать `ref` вместо deps?

## ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useEffect>

## Input лагает при фильтрации большого списка — исправь.

### ОПИСАНИЕ

Каждое нажатие триггерит фильтр огромного массива в render. Решения: `useDeferredValue` (input реагирует, фильтр идёт low-priority), `debounce` запроса, виртуализация результата, `useMemo` для `filter`-функции.

### КАК РАБОТАЕТ

- 01 `useDeferredValue` — UI остаётся отзывчивым.
- 02 `Debounce` для серверного запроса.
- 03 `useMemo(() => filter(items, q), [items, q])`.
- 04 Виртуализация (`react-window`) для отображения.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 `Search-typeahead`.
- 02 `Filter` большой таблицы.
- 03 `Realtime-фильтрация`.

### ПОДВОДНЫЕ КАМНИ

- 01 `filter` в `render` без `useMemo` —  $O(N)$  каждый `keystroke`.
- 02 Без виртуализации — DOM лагает.

### КОД

```
// useDeferredValue
const deferred = useDeferredValue(q);
const list = useMemo(() => filter(items, deferred), [items, deferred]);
```

### МОГУТ ДОСПРОСИТЬ

- 01 Чем `deferred` от `debounce` отличается?
- 02 Когда нужна виртуализация?

### ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useDeferredValue>

ДЕНЬ 2 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

**WS FREEZE****STALE QUOTES****STALE SEARCH****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Real-time проблемы и race conditions.

**ВОПРОСЫ ДНЯ**

Q04 WebSocket UI фризит страницу — исправь.

Q05 Старая котировка перетирает новую — исправь.

Q06 После быстрого поиска показываются результаты старого запроса — исправь.

## WebSocket UI фризит страницу – исправь.

### ОПИСАНИЕ

100+ сообщений в секунду → setState на каждое → render-storm. Решения: батчить (накопить N мс – flush), rAF-batching, отдельный store для high-frequency данных, перенести парсинг в Web Worker, виртуализация UI.

### КАК РАБОТАЕТ

- 01 Накапливать в Map → flush через setTimeout / rAF.
- 02 Web Worker для CPU-интенсивного парсинга.
- 03 Виртуализация для рендера.
- 04 Отдельный store с подпиской через useSyncExternalStore.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Биржевые тики.
- 02 Live-логи.
- 03 Multiplayer-доски.

### ПОДВОДНЫЕ КАМНИ

- 01 setState на каждое сообщение – kill UI.
- 02 Без батчинга – N рендеров.

### КОД

// Батчинг через rAF

```
let pending: Msg[] = [];  
ws.onmessage = (e) => {  
  pending.push(JSON.parse(e.data));  
  if (pending.length === 1) requestAnimationFrame(flush);  
};  
function flush() { store.applyBatch(pending); pending = []; }
```

### МОГУТ ДОСПРОСИТЬ

- 01 Чем rAF от setTimeout отличается?
- 02 Когда выносить в Web Worker?

### ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame>



## ВОПРОС Q05

**Старая котировка перетирает новую – исправь.**

## ОПИСАНИЕ

Race condition: запрос за «ab» вернулся позже «abc» и перезаписал актуальные данные.  
Решения: версионный счётчик (сравнить id с актуальным), AbortController (отменить старый), маркер последнего ввода.

## КАК РАБОТАЕТ

- 01 lastId++ перед запросом, проверка после ответа.
- 02 AbortController на каждый новый запрос.
- 03 Сравнить ключ ответа с current state.

## ГДЕ ВСТРЕЧАЕТСЯ

- 01 Search input.
- 02 Автоподсказки.
- 03 Live-prices.

## ПОДВОДНЫЕ КАМНИ

- 01 Без id-counter – race в любой момент.
- 02 AbortError путать с network error.

## КОД

// Счётчик версий

```
let lastId = 0;
async function search(q) {
  const myId = ++lastId;
  const data = await api.search(q);
  if (myId !== lastId) return; // устарело
  setResults(data);
}
```

## МОГУТ ДОСПРОСИТЬ

- 01 Чем AbortController от version-counter отличается?
- 02 Что такое stale response?

## ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/AbortController>

## После быстрого поиска показываются результаты старого запроса – ис

### ОПИСАНИЕ

То же race condition (см. Q5) – но в React-эффекте. Решения: cleanup-флаг (cancelled = true в return useEffect → перед setState проверить), AbortController в useEffect, переход на react-query (закрывает race из коробки).

### КАК РАБОТАЕТ

- 01 useEffect(() => { let cancelled = false; ... return () => { cancelled = true; }; }).
- 02 Перед setState – if (!cancelled).
- 03 AbortController + cleanup ctrl.abort().
- 04 react-query: queryKey включает q → корректный кэш.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Search в React.
- 02 Дашборды с динамическими props.

### ПОДВОДНЫЕ КАМНИ

- 01 Забыть cleanup → setState на unmounted.
- 02 AbortError throw – обработать в catch.

### КОД

// Cleanup-флаг

```
useEffect(() => {
  let cancelled = false;
  fetchUsers(q).then(d => { if (!cancelled) setUsers(d); });
  return () => { cancelled = true; };
}, [q]);
```

### МОГУТ ДОСПРОСИТЬ

- 01 Чем react-query упрощает?
- 02 Когда AbortController, когда флаг?

### ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest>

ДЕНЬ 3 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

MEMORY LEAK

MODAL KEYBOARD

DROPDOWN A11Y

## ЧТО УЧИМ В ЭТОТ ДЕНЬ

Утечки и accessibility-баги.

## ВОПРОСЫ ДНЯ

Q07 Memory leak после ухода со страницы – найди.

Q08 Modal ломает keyboard navigation – исправь.

Q09 Dropdown недоступен для screen reader – исправь.

## Memory leak после ухода со страницы – найди.

### ОПИСАНИЕ

Симптомы: heap растёт, detached DOM nodes остаются. Источники: подписки без unsubscribe (WebSocket, addEventListener, setInterval), замыкания над большими данными, ref на DOM в массиве. Дебаг через DevTools Memory → Heap snapshot.

### КАК РАБОТАЕТ

- 01 Heap snapshot перед / после unmount.
- 02 Diff: что осталось.
- 03 Retainer chain → кто держит.
- 04 Cleanup: removeEventListener / clearInterval / unsubscribe.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 SPA с долгими сессиями.
- 02 После refactor.
- 03 Регрессии.

### ПОДВОДНЫЕ КАМНИ

- 01 DevTools удерживает свои ссылки – закрыть Console.
- 02 Strict Mode dev двойные подписки → шум в snapshot.

### КОД

```
// Cleanup
```

```
useEffect(() => {  
  const t = setInterval(tick, 1000);  
  return () => clearInterval(t);  
}, []);
```

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое detached DOM?
- 02 Чем WeakMap помогает?

### ПОЛНАЯ СТАТЬЯ

<https://developer.chrome.com/docs/devtools/memory-problems>

## ВОПРОС Q08

**Modal ломает keyboard navigation – исправь.**

## ОПИСАНИЕ

Проблема: Tab уходит за модалку, focus теряется при закрытии. Решения: focus-trap (зациклить Tab внутри модалки), запомнить trigger-элемент при открытии и вернуть focus при закрытии, использовать <dialog> или react-aria/focus-trap-react.

## КАК РАБОТАЕТ

- 01 При open: запомнить document.activeElement.
- 02 Focus first focusable внутри modal.
- 03 Tab → next focusable; на последнем → first.
- 04 При close: вернуть focus на trigger.
- 05 Native <dialog> делает многое автоматически.

## ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая custom модалка.
- 02 Drawer / sidebar.
- 03 Combobox.

## ПОДВОДНЫЕ КАМНИ

- 01 Не вернуть focus → пользователь теряется.
- 02 Скрытый UI остался accessible через Tab.

## МОГУТ ДОСПРОСИТЬ

- 01 Что делает <dialog>?
- 02 Что такое inert?

## ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/dialog>

## Dropdown недоступен для screen reader – исправь.

### ОПИСАНИЕ

Проблемы: div вместо button, нет aria-expanded, нет связи trigger ↔ menu, нет keyboard navigation (Arrow keys, Esc). Решения: native <button>, role="listbox" + role="option", aria-expanded / aria-controls, keyboard events. Лучше – взять headless-UI (radix, react-aria).

---

### КАК РАБОТАЕТ

- 01 <button aria-haspopup="listbox" aria-expanded={open}>.
- 02 <ul role="listbox" id={menuId}>.
- 03 ArrowDown / ArrowUp / Enter / Esc.
- 04 aria-activedescendant для virtual focus.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Custom select / dropdown.
- 02 Combobox / autocomplete.
- 03 Action menu.

---

### ПОДВОДНЫЕ КАМНИ

- 01 div + onClick – недоступно.
- 02 Только mouse-events.
- 03 ARIA без semantics – screen reader путается.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое headless-UI?
- 02 Какие keyboard-shortcuts в combobox?

---

### ПОЛНАЯ СТАТЬЯ

<https://www.radix-ui.com/primitives/docs/components/dropdown-menu>

ДЕНЬ 4

НАГРУЗКА

СРЕДНИЙ

3 ВОПРОСА

TABLE FOCUS

REACT KEY

CONTEXT RE-RENDER

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Фокус, ключи, контекст-перформанс.

ВОПРОСЫ ДНЯ

Q10 Table теряет focus при sort — исправь.

Q11 React key bug ломает список — исправь.

Q12 Context вызывает re-render всего приложения — исправь.

## Table теряет focus при sort – исправь.

### ОПИСАНИЕ

При sort React пересоздаёт строки (без стабильных keys). Focus теряется. Решения: использовать стабильный key (id), сохранить focused-id перед sort, восстановить focus после re-render через useEffect.

---

### КАК РАБОТАЕТ

- 01    Key должен быть row.id, не index.
- 02    Перед sort: запомнить activeElement / row-id.
- 03    После: el.focus().
- 04    Можно через ref + useEffect.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01    Sortable / filterable tables.
- 02    Динамические списки.

---

### ПОДВОДНЫЕ КАМНИ

- 01    key=index → React пересоздаёт DOM → focus уходит.
- 02    Auto-focus first row после sort – раздражает.

---

### МОГУТ ДОСПРОСИТЬ

- 01    Почему key важен?
- 02    Как сохранять focus в виртуализированной таблице?

---

### ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/rendering-lists#why-does-react-need-keys>



## ВОПРОС Q11

**React key bug ломает список – исправь.**

## ОПИСАНИЕ

Симптомы: input перепутаны, state «уехал» после reorder. Причина: key=index, или Math.random, или ничего. Решение: стабильный уникальный key (обычно id) – React корректно матчит компоненты после reorder.

## КАК РАБОТАЕТ

- 01 key={item.id} – стабильный.
- 02 key=index – bug при insert/reorder.
- 03 key=Math.random() – пересоздаётся каждый рендер.
- 04 Если нет id – генерировать (uuid) при создании.

## ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой .map() в JSX.
- 02 Сортировка / фильтрация.
- 03 Drag-drop.

## ПОДВОДНЫЕ КАМНИ

- 01 Дубли key → warning + баги.
- 02 Index как key выглядит «работает», ломается при reorder.

## МОГУТ ДОСПРОСИТЬ

- 01 Когда index безопасен?
- 02 Что такое stable key?

## ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/rendering-lists>

## Context вызывает re-render всего приложения – исправь.

### ОПИСАНИЕ

Каждое изменение `Context.value` перерисовывает всех consumer'ов. Решения: split Context (state и actions отдельно), `useMemo` для value, `use-context-selector` (granular), перевод на Zustand / Redux с селекторами.

---

### КАК РАБОТАЕТ

- 01 `Inline value={{ a, b }}` – новая ссылка каждый рендер.
- 02 `useMemo: const value = useMemo(() => ({a, b}), [a, b])`.
- 03 Split: `StateCtx` + `DispatchCtx`.
- 04 `use-context-selector` – подписка на slice.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Большой Context часто меняется.
- 02 Дашборды с глобальным state.

---

### ПОДВОДНЫЕ КАМНИ

- 01 `useMemo` с `inline-deps` – всё равно новая ссылка.
- 02 `React.memo` не спасает от Context-rerender.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Чем split state/dispatch помогает?
- 02 Когда брать Zustand?

---

### ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useContext#optimizing-re-renders-when-passing-objects-and-functions>

ДЕНЬ 5 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

**useMemo НЕ ПОМОГАЕТ BUNDLE +500KB PLAYWRIGHT FLAKY**

## ЧТО УЧИМ В ЭТОТ ДЕНЬ

Performance и стабильность тестов.

## ВОПРОСЫ ДНЯ

- Q13 useMemo не помогает performance – объясни и исправь.
- Q14 Bundle внезапно вырос на 500KB – найди причину.
- Q15 Playwright test flaky – найди причину.

## useMemo не помогает performance – объясни и исправь.

### ОПИСАНИЕ

Возможные причины: deps нестабильные (объект/функция пересоздаются), маленький calc (overhead useMemo больше выгоды), useMemo с немемоизированным результатом, проблема не в мемо а в re-render всего дерева сверху.

---

### КАК РАБОТАЕТ

- 01 Профайлер: пересчитывается ли useMemo каждый раз?
- 02 Логировать deps – стабильны ли?
- 03 Если calc дешёвый – убрать useMemo.
- 04 Решить вышестоящий ререндер (React.memo на parent).

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 После добавления useMemo нет улучшения.
- 02 Reactor performance regression.

---

### ПОДВОДНЫЕ КАМНИ

- 01 useMemo на простую арифметику – overhead больше выгоды.
- 02 Inline-объект в deps → useMemo бесполезен.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Когда useMemo – антипаттерн?
- 02 Чем React.memo от useMemo отличается?

---

### ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useMemo>

## ВОПРОС Q14

**Bundle внезапно вырос на 500KB – найди причину.**

## ОПИСАНИЕ

Шаги: сравнить bundle-analyzer до и после, проверить новые dependency (npm-diff), проверить, не подтянулась ли тяжёлая транзитивная (lodash, moment), отключённый tree-shake (CommonJS-import). Решение: переход на lightweight аналог, dynamic import, removal.

## КАК РАБОТАЕТ

- 01 webpack-bundle-analyzer / rollup-plugin-visualizer.
- 02 git diff package-lock.json.
- 03 size-limit / bundlewatch в CI.
- 04 bundlephobia.com для проверки npm.

## ГДЕ ВСТРЕЧАЕТСЯ

- 01 После обновления deps.
- 02 Регулярные аудиты.
- 03 Lighthouse регрессия.

## ПОДВОДНЫЕ КАМНИ

- 01 Без analyzer – ищем вслепую.
- 02 Tree-shake не работает для CommonJS.
- 03 Дубли версий в monorepo.

## МОГУТ ДОСПРОСИТЬ

- 01 Что такое tree-shake?
- 02 Чем bundlephobia полезен?

## ПОЛНАЯ СТАТЬЯ

<https://github.com/webpack-contrib/webpack-bundle-analyzer>

## Playwright test flaky – найди причину.

### ОПИСАНИЕ

Flaky = иногда зелёный, иногда красный. Причины: race с анимациями, network-латентность, timing-зависимость (setTimeout вместо waitFor), отсутствие test isolation, общие test data. Решения: locator с auto-waiting, explicit waits, изоляция данных, mocking сети.

---

### КАК РАБОТАЕТ

- 01 Заменить sleep на toHaveText / toBeVisible.
- 02 getByRole + auto-waiting.
- 03 Mock network через route.
- 04 Чистить state между тестами.
- 05 test.describe.parallel(false) для изоляции.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой нестабильный тест.
- 02 После изменения UI.
- 03 После добавления async-действий.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Хардкод waitForTimeout(1000) – flaky.
- 02 Тесты делят DB / cookies → race.
- 03 CSS-селекторы хрупкие.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое auto-waiting?
- 02 Чем page.route помогает?

---

### ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/best-practices>

ДЕНЬ 6 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

HYDRATION MISMATCH

PROD-ONLY ERROR

CORS

ЧТО УЧИМ В ЭТОТ ДЕНЬ

SSR / build-only / network проблемы.

ВОПРОСЫ ДНЯ

Q16 Hydration mismatch – найди причину.

Q17 Ошибка появляется только в production build – расследуй.

Q18 CORS request падает – объясни возможные причины.

## Hydration mismatch – найди причину.

### ОПИСАНИЕ

Server и client отрендерили разный HTML. Причины: `Date.now` / `Math.random` в `render`, `typeof window` check, пользовательская локаль / `theme detection`. Решения: двухпроходный рендер (`placeholder` на сервере → реальное в `useEffect`), `suppressHydrationWarning` точно, `client-only`-компонент через `dynamic({ ssr: false })`.

---

### КАК РАБОТАЕТ

- 01 В `render`: `Date.now()/Math.random()/typeof window/localStorage`.
- 02 `useEffect` только на клиенте – туда динамику.
- 03 `suppressHydrationWarning` – для конкретного `<element>`.
- 04 Next.js `dynamic ssr:false` для `full-client`.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 SSR / Next / Remix.
- 02 Theme detection.
- 03 Localized dates.

---

### ПОДВОДНЫЕ КАМНИ

- 01 `suppressHydrationWarning` – скрывание, не решение.
- 02 Flash of wrong content при двухпроходе.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое flash of wrong content?
- 02 Чем `server component` помогает?

---

### ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react-dom/client/hydrateRoot>



## ВОПРОС Q17

**Ошибка появляется только в production build – расследуй.**

## ОПИСАНИЕ

Возможно: minification поломала тяжёлый код / dynamic имена, NODE\_ENV=production отключил dev-проверки, разные polyfills, source-maps выключены – нужно включить, Strict Mode dev-warnings были скрыты.

## КАК РАБОТАЕТ

- 01 Включить source-maps в production build.
- 02 Sourcemap → найти строку реальную.
- 03 Reproduce: build + serve локально, не dev-server.
- 04 Сравнить .env.production vs .env.development.

## ГДЕ ВСТРЕЧАЕТСЯ

- 01 Sentry-репорты только из prod.
- 02 Регрессии после deploy.

## ПОДВОДНЫЕ КАМНИ

- 01 Без source-maps stack trace бесполезен.
- 02 process.env.NODE\_ENV branches скрывают.

## МОГУТ ДОСПРОСИТЬ

- 01 Почему source-maps публиковать осторожно?
- 02 Как воспроизвести prod локально?

## ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/source-maps>

## CORS request падает – объясни возможные причины.

### ОПИСАНИЕ

Браузер блокирует cross-origin response, если Access-Control-Allow-Origin не разрешает. Причины: сервер не вернул нужные headers, preflight (OPTIONS) не прошёл, credentials:true без правильного Allow-Credentials, неподдерживаемый Origin / Method / Header.

---

### КАК РАБОТАЕТ

- 01 Network tab: preflight OPTIONS статус.
- 02 Server должен вернуть Access-Control-Allow-Origin.
- 03 credentials:'include' требует Allow-Credentials:true и точный Origin.
- 04 Custom headers требуют Allow-Headers.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Frontend на одном домене, API на другом.
- 02 API Gateway misconfig.
- 03 После переноса домена.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Wildcard \* + credentials → не работает.
- 02 Backend не отвечает на OPTIONS.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое preflight?
- 02 Почему CORS не security?

---

### ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

ДЕНЬ 7

НАГРУЗКА

ТЯЖЁЛЫЙ

2 ВОПРОСА

## XSS TOKEN REFRESH RACE

### ЧТО УЧИМ В ЭТОТ ДЕНЬ

Безопасность и race в auth.

### ВОПРОСЫ ДНЯ

Q19 XSS vulnerability в preview HTML — исправь.

Q20 Race condition в token refresh — исправь.

## XSS vulnerability в preview HTML – исправь.

### ОПИСАНИЕ

User-content рендерится через dangerouslySetInnerHTML без sanitization → XSS. Решение: DOMPurify.sanitize(html) перед рендером, или использовать markdown-renderer с whitelist, или не рендерить HTML вообще. CSP – дополнительная защита.

---

### КАК РАБОТАЕТ

- 01    DOMPurify.sanitize(html, options).
- 02    Whitelist tags + attrs.
- 03    CSP с nonce / hash.
- 04    Если возможно – использовать text rendering, не innerHTML.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01    User comments, posts, messages.
- 02    Markdown / WYSIWYG.
- 03    Email preview.

---

### ПОДВОДНЫЕ КАМНИ

- 01    Без sanitize – script injection.
- 02    Whitelist слишком широкий – onerror= атрибут.
- 03    Doc.title тоже может быть XSS.

---

### КОД

```
// DOMPurify
import DOMPurify from 'dompurify';
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(html) }} />
```

---

### МОГУТ ДОСПРОСИТЬ

- 01    Чем CSP помогает против XSS?
- 02    Что такое stored vs reflected XSS?

---

### ПОЛНАЯ СТАТЬЯ

<https://github.com/cure53/DOMPurify>

## ВОПРОС Q20

**Race condition в token refresh – исправь.**

## ОПИСАНИЕ

Несколько параллельных запросов получили 401, каждый запустил refresh → дубль refresh, токены конфликтуют. Решение: singleton refresh-промис (если идёт refresh – ждать его), очередь pending requests, retry после refresh.

## КАК РАБОТАЕТ

```
01 let refreshPromise: Promise<Token> | null = null.  
02 if (refreshPromise) return refreshPromise.  
03 После complete – null.  
04 Pending requests subscribe → resume после refresh.
```

## ГДЕ ВСТРЕЧАЕТСЯ

```
01 JWT с коротким TTL.  
02 Параллельные fetch на странице.  
03 Auto-refresh middleware.
```

## ПОДВОДНЫЕ КАМНИ

```
01 Без singleton → N refresh → backend rate-limit.  
02 Race с logout – refresh уже невалидный.
```

## КОД

```
// Singleton refresh
```

```
let refresh: Promise<Token> | null = null;  
async function getValidToken() {  
  if (!refresh) refresh = api.refresh().finally(() => refresh = null);  
  return refresh;  
}
```

## МОГУТ ДОСПРОСИТЬ

```
01 Что такое axios interceptor?  
02 Где хранить refresh-token?
```

## ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc6749#section-6>

# СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Найти и исправить infinite render / effect, lag input (Q1, Q2, Q3)

---

- ☐ Исправить WS freeze, stale quotes, stale search (Q4, Q5, Q6)

---

- ☐ Найти memory leak, исправить modal keyboard и dropdown ally (Q7, Q8, Q9)

---

- ☐ Исправить table focus, key bug, context re-render (Q10, Q11, Q12)

---

- ☐ Объяснить почему useМемо не помогает, найти bundle bloat, fix flaky test (Q13, Q14, Q15)

---

- ☐ Найти hydration mismatch, prod-only error, объяснить CORS (Q16, Q17, Q18)

---

- ☐ Исправить XSS и race в token refresh (Q19, Q20)

---

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА [learn.javascript.ru](https://learn.javascript.ru)